

Binary Numbers

ACOL215: Digital Logic And System Design
Chapter 1

Abhilash Jindal

Digital systems

- Represent and manipulate *discrete elements* (set) of information. Example: *digits (hence digital systems)*, letters of alphabet, playing cards, chessboards squares
- Discrete elements are represented by electrical signals (voltages/current)
- Signals are controlled and manipulated by transistors

Discrete quantities of information

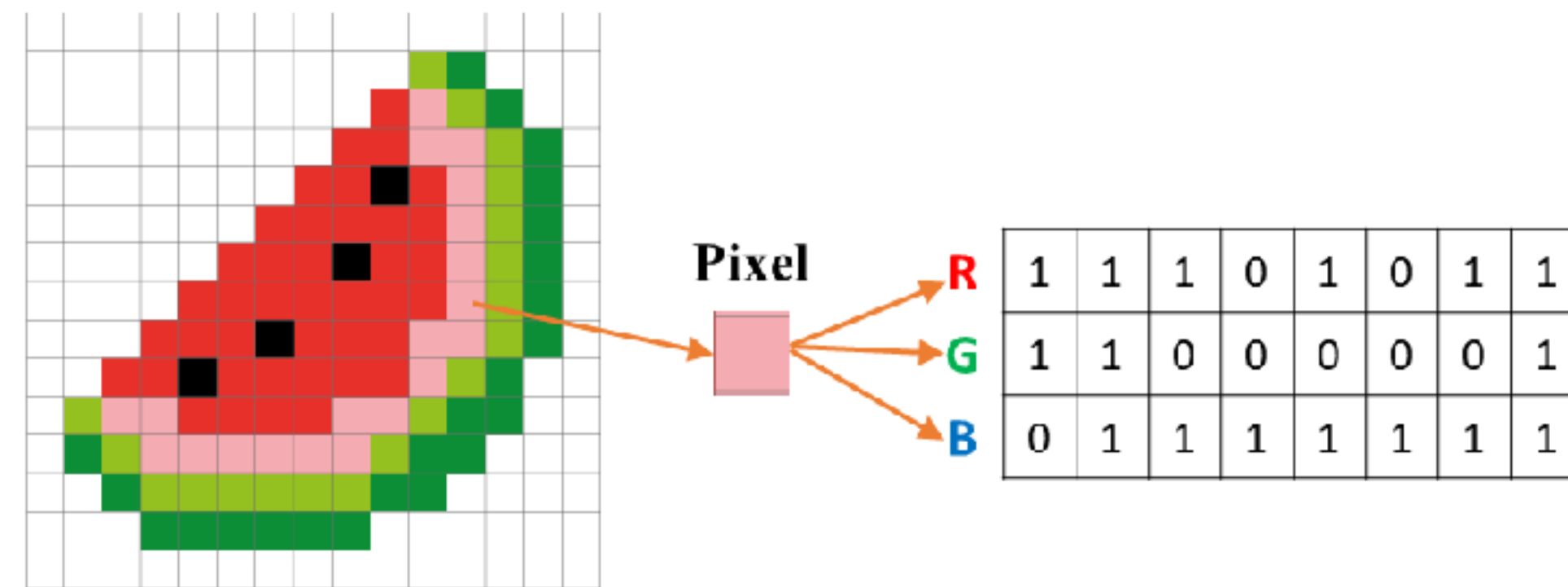
- Inherently discrete processes
 - Payroll management: Employee names, PAN Card number, monthly salaries, income tax deduction
- Continuous processes
 - Example: Digital camera.
 - Colors are discretised into numbers
 - Images are discretised into pixels, where each pixel has a discretised color

	Red	Green	Blue
	0	0	0
	255	255	255
	255	0	0
	0	255	0
	0	0	255
	255	128	0
	255	255	0
	128	128	128



Present-day digital systems

- Use just two discrete values: on<>off
- Binary digit (bit) has two numerical values: 0 or 1
- Discrete elements of information are represented with group of bits



Decimal numbers

How to represent numbers with bits?

- Consider 7,392.54
- $= 7000 + 300 + 90 + 2 + 0.5 + 0.04$
- $= 7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0 + 5 \times 10^{-1} + 4 \times 10^{-2}$
- In general, a number $\cdots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \cdots =$
 $\cdots + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots$
- For 7,392: $a_3 = 7; a_2 = 3; a_1 = 9; a_0 = 2$. Rest are zero.

Radix

- A decimal number $\cdots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \cdots = \cdots + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots$
- Decimal numbers have **10** symbols: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}.
- Each coefficient is multiplied with a power of **10**.
- Number is in base 10 or radix 10
- Written as $(7,392.54)_{10}$

Base-r number system

- Base-r numbers have **r** symbols: $\{0, \dots, r - 1\}$. $a_j \in \{0, \dots, r - 1\}$
- A base-r number $\dots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \dots =$
 $\dots + a_3 \times r^3 + a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2} + \dots$
- Each coefficient is multiplied with a power of **r**.

Examples

- Base-5 numbers have 5 symbols $\{0, \dots, 4\}$. $(4021.2)_5 = ?$
 - $4 * 5^3 + 0 * 5^2 + 2 * 5^1 + 1 * 5^0 + 2 * 5^{-1} = (511.4)_{10}$
- Octal numbers (base-8) have 8 symbols $\{0, \dots, 7\}$. $(127.4)_8$
 - $1 * 8^2 + 2 * 8^1 + 7 * 8^0 + 4 * 8^{-1} = (87.5)_{10}$

Binary numbers

- Binary numbers (base-2) have 2 symbols $\{0,1\}$. $(11010.11)_2 = ?$
 - $1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 + 1 * 2^{-1} + 1 * 2^{-2} = (26.75)_{10}$
- $(10101)_2 = ?$
 - $(21)_{10}$
- $(110101)_2 = ?$
 - $(53)_{10}$

Hexadecimal numbers

- Hexadecimal numbers (base-16) have 16 symbols $\{0, \dots, 9, A, B, C, D, E, F\}$.
 $(B65F)_{16} = ?$
 - $11 * 16^3 + 6 * 16^2 + 5 * 16^1 + 15 * 16^0 = (46,687)_{10}$

Arithmetic operations on base-r numbers

Addition

- $101101 + 100111 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- When sum is $2_{10} (=10_2)$, write 0 and carry 1.
When sum is $3_{10} (=11_2)$, write 1 and carry 1
- $(101101)_2 = (45)_{10}$, $(100111)_2 = (39)_{10}$, $(1010100)_2 = (84)_{10}$

Arithmetic operations on base-r numbers

Subtraction

- $101101 - 100111 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- In decimal, say 96-47.
 - When we encounter “6-7”, put (10-1) in output and reduce previous digit by 1, i.e., 9 to 8.
- In binary, 0-0 is 0. 1-1 is 0. 1-0 is 1.
When encountering 0-1: write (2-1) in output and reduce previous bit by 1.
- $(101101)_2 = (45)_{10}$, $(100111)_2 = (39)_{10}$, $(000110)_2 = (6)_{10}$

Arithmetic operations on base-r numbers

Multiplication

- $1011 * 101 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- When multiplier digit is 1, copy multiplicand in the partial product. Shift multiplicand left for every bit. Compute addition of all partial products
- $(1011)_2 = (11)_{10}$, $(101)_2 = (5)_{10}$, $(110111)_2 = (55)_{10}$

N-bit binary numbers

- 3-digit decimal numbers can represent 10^3 numbers: $\{000, 001, \dots, 999\}$
- 3-bit binary numbers can represent 2^3 numbers: $\{000, 001, \dots, 111\}$ i.e., $\{0, \dots, 7\}$
- A byte has 8-bits. It can represent 2^8 numbers: $\{0, \dots, 255\}$. Or $\{0, \dots, FF\}$ in hexadecimal.
 - Enough to represent a keyboard character

Kilo-Mega-Giga-Tera

Larger groups of bits

- Kilo (K): 2^{10} , Mega (M): 2^{20} , Giga (G): 2^{30} , Tera (T): 2^{40}
- 1MB of file contains 2^{20} bytes = ~1 Million bytes.
How many *bits* can 1TB of disk store?

Table 1.1
Powers of Two

<i>n</i>	<i>2ⁿ</i>	<i>n</i>	<i>2ⁿ</i>	<i>n</i>	<i>2ⁿ</i>
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1,024 (1K)	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096 (4K)	20	1,048,576 (1M)
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

Characters

ASCII character code

- 01/char.c

Table 1.7
American Standard Code for Information Interchange (ASCII)

<i>b₄b₃b₂b₁</i>	<i>b₇b₆b₅</i>							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	‘	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L		l	
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

Error-Detecting Code

- ASCII is a 7-bit code. 8th *parity bit* is sometimes added to check for transmission/storage retrieval errors.
- Odd parity: Parity bit is **0** if number of 1s are **odd**, 1 otherwise
- Even parity: Parity bit is **0** if number of 1s are **even**, 1 otherwise

		Even parity	Odd parity
ASCII A	1000001	01000001	11000001
ASCII T	1010100	11010100	01010100

- What is the even parity of 0101100?

Converting decimal integers to binary

- How to find coefficients $\cdots a_2 a_1 a_0 . a_{-1} a_{-2} \cdots = \cdots + a_2 \times 2^2 + a_1 \times 2^1 + a_0 \times 2^0 + a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \cdots$
- Continue dividing the number and all successive quotients by 2 until the quotient reaches 0
- $(41)_{10} = ?$
- $(101001)_2$

	Quotient	Remainder
41/2	20	1
20/2	10	0
10/2	5	0
5/2	2	1
2/2	1	0
1/2	0	1



Converting decimal integers to octal

- $(153)_{10} = ?$
- $(231)_8$
- $(135)_{10} = (??)_2$
- $(135)_{10} = (??)_8$

	Quotient	Remainder
153/8	19	1
19/8	2	3
2/8	0	2

Converting decimal *fraction* to binary

- Multiply the number and each successive fraction with 2 at each step, until the fraction reaches 0 or until enough significant coefficients are obtained
- $(0.6875)_{10} = ?$
- $(0.1011)_2$

	Integer	Fraction
$0.6875 * 2$	1	0.3750
$0.3750 * 2$	0	0.75
$0.750 * 2$	1	0.5
$0.500 * 2$	1	0



Converting decimal *fraction* to octal

- $(0.513)_{10} = ?$
- $(0.406517..)_{8}$
- Overall: separate integer and fraction parts. Convert them to base-r numbers separately, and recombine them.
- $(153.513)_{10} = (231.406517)_{8}$

	Integer	Fraction
$0.513 * 8$	4	0.104
$0.104 * 8$	0	0.832
$0.832 * 8$	6	0.656
$0.656 * 8$	5	0.248
$0.248 * 8$	1	0.984
$0.984 * 8$	7	0.872
...	...	...

Representing long string of bits

- While computers work with binary numbers, all human interfaces specify long bits using octal/hexadecimal/or even base-256.
- Instead of writing a color of 111111110000000011111111, it is more convenient to write FF00FF

	Red	Green	Blue	Hexadecimal code
	0	0	0	#000000
	255	255	255	#FFFFFF
	255	0	0	#FF0000
	0	255	0	#00FF00
	0	0	255	#0000FF
	255	128	0	#FF8000
	255	255	0	#FFFF00
	128	128	128	#808080

Representing long string of bits

Octal and Hexadecimal numbers

- Shorter patterns of characters are easier to identify than long patterns of 1's and 0's
- Since $2^3 = 8$, each octal number represents 3 bits
- Since $2^4 = 16$, each hexadecimal number represents 4 bits

Decimal	Binary	Octal	Hexadecimal
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Converting binary to octal/hexadecimal

- Partition into groups of 3 bits. Assign the corresponding octal digit to each group
- $(10110001101011.111100000110)_2$ to octal?

10	110	001	101	011	.	111	100	000	110
2	6	1	5	3	.	7	4	0	6

- $(10110001101011.11110010)_2$ to hexadecimal?

10	1100	0110	1011	.	1111	0010
2	C	6	B	.	F	2

Converting octal/hexadecimal to binary

- Reverse the procedure: replace each octal digit with its 3-bit binary equivalent. Each hexadecimal digit with its 4-bit binary equivalent
- $(673.124)_8$ to binary?

6	7	3	.	1	2	4
110	111	011	.	001	010	100

- $(306.D)_{16}$ to binary?

3	0	6	.	D
0011	0000	0110	.	1101

Representing long string of bits (2)

- An IPv4 address is a base-256 number: $(10.237.23.38)_{256}$
- Convert to binary/hexadecimal?
- How many unique IPv4 addresses can be there?